Research Statement

Luís Pina — University of Illinois Chicago

June 19, 2026[1]

My research makes modern software observable, reproducible, and trustworthy. I work at the intersection of Software Engineering, Programming Languages, and Software Systems because reliability failures in modern software rarely belong to only one layer: They arise from complex and subtle interactions between language semantics, managed-runtime behavior, concurrency, libraries, and the execution environment itself. My work builds systems that record and reproduce these interactions (via MVX and RR), and search them for faults (via random testing), with the unifying goal of making complex software, written in managed languages, predictable for developers to test, update, debug, and, ultimately, trust.

Multi-Version Execution and Record/Replay for Managed Languages. The central contribution of my independent research program at UIC, and the area in which I have established myself as a leader, is to turn Multi-Version eXecution (MVX) and Record/Replay (RR) into practical techniques for managed languages. Prior work focused almost exclusively on native C/C++ code, despite the fact that most modern software targets managed runtimes (*e.g.,* Java, JavaScript, Python) that pose new hard problems not addressed by native techniques, such as: Just-In-Time compilation, garbage collection, dynamic code loading, and pervasive multi-threading. MVX runs multiple variants of a program together and compares or coordinates their behavior, with applications to reliability, security, dynamic updating, and availability. RR records an execution so that it can later be replayed deterministically, exposing rare or production-only failures in development environments where they can be fixed.

A design principle of my research is deployability: reliability mechanisms should not require modified runtimes, otherwise they risk staying academic and never reaching practice. The insight behind my work is to relocate the point of control to the managed-language interface itself, thus showing that MVX and RR can be made practical even under this constraint. I demonstrated this idea in two very different and complex managed environments. SINATRA delivers zero-downtime, no-data-loss updates to unmodified browsers, including Firefox and Chrome, by switching users to a freshly updated browser in under 10ms [ECOOP'23]. JMVX shows that fast MVX and RR are possible on an unmodified Java Virtual Machine [OOPSLA'24]. Building on JMVX, my work on Relaxed Total Order reduces the cost of replaying multithreaded executions by recording only the orderings needed for faithful replay [ECOOP'26], lowering recording overhead from 21.7% to 13.7% and replay overhead from 64.9% to 31.7%. Across these systems, I also uncovered and exploited a surprising synergy between MVX and RR: the same combination that prevents data loss during browser updates [ECOOP'23] also reduces the cost of recording I/O-bound workloads from 196.1% to 25.8% [ECOOP'26].

This agenda is supported by my single-PI NSF award, and it anchored the doctoral work of my student David Schwartz, who recently defended his dissertation on fast MVX and RR for managed languages. Together, these results move MVX/RR from a native-code curiosity to a practical capability for the languages in which most modern software is written.

Random Testing and Property-Based Testing for Managed Languages. Besides tolerating and reproducing faults (via MVX and RR), I also focus on finding such faults to start with, in particular via random testing and the design of effective automated testing tools. My most recent work in this area, PropCov, developed with my students Jesse Coultas and Joseph Wiseman, provides a coverage framework for Property-Based Testing [ISSTA'26]. PBT asks developers to state properties about the software, and then generates inputs to refute them. Yet, developers have no principled way to know what a property test actually exercises. Traditional coverage tools are misleading in this setting because they do not distinguish code missed by the generator from code that is infeasible under the property's input domain. In *PropCov*, we defined and measured per-property reachability coverage, built on static-analysis, giving a precise, per-property account of how thoroughly each property is tested, together with concrete suggestions of how to improve existing tests. We evaluated PropCov on 293 properties

[1] Most recent version of this document available online

over 25 projects and found it reduces the code a developer must inspect by 86%. Using its guidance, we found 5 previously unknown bugs and improved 42 property tests across 7 projects.

PropCov connects the practical problem of test adequacy with deeper PL/SE questions about reachability, generators, and the semantics of test inputs, and also represents an independent continuation of my broader interest in randomized testing. It builds on my earlier collaborative work in fuzzing and automated test generation. In CONFETTI, we showed how to combine greybox fuzzing with concolic execution for managed languages [ICSE'22]. In subsequent collaborative work on fuzzer mutator performance, we performed a large-scale study of the mutation operators used by modern fuzzers and showed that seemingly small, low-level design choices can significantly change the coverage and bugs found by a fuzzer [ISSTA'24]. Together, these earlier collaborations shaped my current view that randomized testing needs a principled foundation that explains why generated tests reach some behaviors, miss others, and expose particular classes of failures.

Reproducibility. Reproducibility is a cross-cutting value in my research. My work makes strong empirical claims about systems that are complex, nondeterministic, and difficult to evaluate. I consider research artifacts (which can reproduce all claims in a paper) not as supplementary material but as part of the research contribution, and I have released such artifacts for all the papers listed in this document. I have also helped build the infrastructure that makes such artifacts credible, serving as co-chair of the Artifact Evaluation Committee for PLDI 2020 and 2021. In collaboration with my UIC colleague Natalie Parde, I studied reproducibility outside my immediate field and co-authored a paper on reproducibility in computational linguistics [EMNLP'22]. We found that source-code availability alone is insufficient and argued for self-contained artifacts and artifact evaluation venues. This perspective also informs how I train students: they should build systems, evaluate them rigorously, and make their claims repeatable by others.

Future directions. In the future, I plan to unify my research directions.

Directed concurrency testing. I plan to close the loop from PropCov's coverage to JMVX's reproducible concurrent bugs. First, I will use per-property reachability as the unit of analysis that filters a noisy static race analysis (*e.g.,* RacerD) and directs a concurrent property-based test, using lightweight schedule perturbation, to pressure the program points where races are both flagged and actually reachable. Then, I will use JMVX to record any violation so it can be deterministically replayed, diagnosed, and shrunk long after it surfaced. This planned work will compose PropCov and JMVX into a single new important capability that existing concurrency-testing tools lack.

Validating LLM-translated code. Translating codebases between languages with LLMs is now common and quietly dangerous: semantic differences between source and target produce bugs that otherwise look correct line by line. Preliminary work from my lab, currently under submission [PYTHONMM], characterizes the conditions under which Python programs are sequentially consistent, which is not an implementation by-product but a guarantee that the standard Python VM quietly provides. Our preliminary work resulted in finding a concurrency-related bug in CPython, which was confirmed and backported to version 3.13. Java offers no such guarantee, so an LLM translating Python to Java can silently drop a property the original depended on (*e.g.,* not declaring a shared variable as `volatile` or not using a `ConcurrentHashMap`). I plan to extend MVX to run variants written in different managed languages, thus enabling the execution of the original and translated program side-by-side, and flagging divergences as candidate defects, turning a class of otherwise invisible translation bugs into observable, reproducible failures.

These directions share the signature of everything I do: build the mechanism, characterize it precisely, back every claim with a reproducible artifact, and aim it at faults that matter. The work is recent and forms a coherent program I am positioned to sustain and grow in the years ahead.